



البرمجة بلغة ++C

جنى الخطاب

User-Defined Functions

Introduction

-Functions are like building blocks.

- They allow complicated programs to be divided into manageable pieces
- Some advantages of functions:
 - A programmer can focus on just that part of the program and construct it, debug it, and perfect it
 - Different people can work on different functions simultaneously
 - Can be re-used (even in different programs)
 - Enhance program readability

ال functions يسمحوا للبرامج المعقدة ان تكون أبسط وأسهل للقراءة والتعديل ويمكن أن يعاد استخدامها داخل البرنامج اكثر من مرة (أو حتى في برامج أخرى) وأكثر من شخص بقدرها يشتغلوا على functions مختلفة بنفس الوقت.

- Functions
 - Called modules
 - Like miniature programs
 - Can be put together to form a larger program

مثل وحدات أو برامج مصغرة، و لما يجتمعوا بشكلوا برنامج كبير.

Predefined Functions

- In algebra, a function is defined as a rule or correspondence between values, called the function's arguments, and the unique value of the function associated with the arguments.

في علم الجبر ال function هو قاعدة أو علاقة بين وهي ال arguments (المعاملات) والقيمة المرتبطة بهذه المعاملات.

– If $f(x) = 2x + 5$, then $f(1) = 7$, $f(2) = 9$, and $f(3) = 11$

- 1, 2, and 3 are arguments

- 7, 9, and 11 are the corresponding values

الarguments هي القيم التي نعطيها الى الfunction ، الcorresponding values هي القيمة التي يعطيها الfunction.

- Some of the predefined mathematical functions are:

`sqrt(x)`

`pow(x, y)`

`floor(x)`

أمثلة على functions رياضية معرفة مسبقا من البرمجية.

- `sqrt(x)` calculates the nonnegative square root of x , for $x \geq 0.0$

– Type double

ال `sqrt(x)` هي اختصار ل square root هذا الfunction لما نستدعيه ونعطيها قيمة برجعنا الجذر التربيعي لها، طبعا القيمة لازم تكون اكبر من صفر، نوع ال argument هون double لكن مش معناه اني ما بقدر ابعت رقم int لانه لو مثلا اعطيته 4 بيعتبرها 4.0 (automatic casting). مثال:

– `sqrt(2.25)` is 1.5 (لاحظوا نوع القيمة اللي رجعتها)

- `pow(x,y)` calculates x to the power y

– Returns a value of type double

– x and y are the parameters (or arguments)

- The function has two parameters

ال `pow` هي اختصار ل power بعطيه (parameters) two arguments واحد للرقم والثاني للقوة برجعلي قيمة الاول مرفوعة لقوة الثاني والنوع اللي برجعه هو double. مثال:

– $\text{pow}(2, 3) = 8.0$ (لاحظوا نوع القيمة التي رجعها).

- The floor function $\text{floor}(x)$ calculates largest whole number not greater than x

– Type double

– Has only one parameter

ال floor function يرجع أكبر عدد صحيح ليس أكبر من القيمة الاصلية التي اعطيناه اياها، والقيمة التي يرجعها تكون من نوع double . مثال:

– $\text{floor}(48.79)$ is 48.0.

- Predefined functions are organized into separate libraries

ال functions المعرفة مسبقا من البرمجية تكون مضمونة في libraries مختلفة.

- Math functions are in cmath header

ال functions الرياضية معرفة في cmath header يعني لما بدنا نستخدم واحد منهم لازم نعمل الها inclusion:

`#include<cmath>`

الأثر الجميل يعمل بصمت

Function	Header File	Purpose	Parameter(s) Type	Result
<code>abs (x)</code>	<code><cstdlib></code>	Returns the absolute value of its argument: <code>abs (-7) = 7</code>	<code>int</code>	<code>int</code>
<code>ceil (x)</code>	<code><cmath></code>	Returns the smallest whole number that is not less than x: <code>ceil (56.34) = 57.0</code>	<code>double</code>	<code>double</code>
<code>cos (x)</code>	<code><cmath></code>	Returns the cosine of angle x: <code>cos (0.0) = 1.0</code>	<code>double</code> (radians)	<code>double</code>
<code>exp (x)</code>	<code><cmath></code>	Returns e^x , where $e = 2.718$: <code>exp (1.0) = 2.71828</code>	<code>double</code>	<code>double</code>
<code>fabs (x)</code>	<code><cmath></code>	Returns the absolute value of its argument: <code>fabs (-5.67) = 5.67</code>	<code>double</code>	<code>double</code>

Function	Header File	Purpose	Parameter(s) Type	Result
<code>floor (x)</code>	<code><cmath></code>	Returns the largest whole number that is not greater than x: <code>floor (45.67) = 45.00</code>	<code>double</code>	<code>double</code>
<code>pow (x, y)</code>	<code><cmath></code>	Returns x^y ; If x is negative, y must be a whole number: <code>pow (0.16, 0.5) = 0.4</code>	<code>double</code>	<code>double</code>
<code>tolower (x)</code>	<code><cctype></code>	Returns the lowercase value of x if x is uppercase; otherwise, returns x	<code>int</code>	<code>int</code>
<code>toupper (x)</code>	<code><cctype></code>	Returns the uppercase value of x if x is lowercase; otherwise, returns x	<code>int</code>	<code>int</code>

abs و fabs يرجعوا القيمة المطلقة الفرق بينهم بس بال data type :

$\text{abs}(-4) = 4$

$\text{fabs}(8.4) = 8.4$

الفرق بين floor و ceil :

floor ترجع أكبر عدد صحيح ليس أكبر من العدد الأصلي

ceil ترجع أكبر عدد صحيح أكبر من العدد الأصلي

بالنسبة ل toupper و tolower ذكرت سابقا ان ال uppercase هي ال capital letter وال lowercase هي small letter، بالتالي toupper بتعطيني ال uppercase من ال character وال tolower بترجع ال lowercase من ال character.

مهم تنتبهوا في toupper و tolower انه ال data type هو int يعني برجع ال ASCII value لل character .

مباركه

بصالح

الأثر الجميل يعمل بصحة

EXAMPLE 6-1

```
//How to use predefined functions.
#include <iostream>
#include <cmath>
#include <cctype>
#include <cstdlib>

using namespace std;

int main()
{
    int    x;
    double u, v;

    cout << "Line 1: Uppercase a is "
         << static_cast<char>(toupper('a'))
         << endl; //Line 1

    u = 4.2; //Line 2
    v = 3.0; //Line 3
    cout << "Line 4: " << u << " to the power of "
         << v << " = " << pow(u, v) << endl; //Line 4

    cout << "Line 5: 5.0 to the power of 4 = "
         << pow(5.0, 4) << endl; //Line 5

    u = u + pow(3.0, 3); //Line 6
    cout << "Line 7: u = " << u << endl; //Line 7

    x = -15; //Line 8
    cout << "Line 9: Absolute value of " << x
         << " = " << abs(x) << endl; //Line 9

    return 0;
}
```

The output:

```
Line 1: Uppercase a is A
Line 4: 4.2 to the power of 3 = 74.088
Line 5: 5.0 to the power of 4 = 625
Line 7: u = 31.2
Line 9: Absolute value of -15 = 15
```

في line 1 استخدمنا static_cast عشان يطبعها A بدون استخدامه كان رح يطبع ASCII value اله.

- To use these functions you must:
 - Include the appropriate header file in your program using the include statement
 - Know the following items:
- Name of the function

- Number of parameters, if any
- Data type of each parameter
- Data type of the value returned: called the type of the function

لحتى نقدر نستخدم هاي ال functions في البرنامج لازم نعمل inclusion لل header file المناسب ونكون عارفين اسم ال function وعدد ال arguments ان وجد وال data type لل parameters والقيمة اللي رح يرجعها. (كل هاي المعلومات موجودين في الجدول فوق).

User-Defined Functions

كل اللي حكينا عنهم قبل هم ال pre-defined functions اللي هي معرفة مسبقا من البرمجية ، الان رح نحكي عن user-defined functions الالمستخدم بكتبتها، وهي نوعين:

- Value-returning functions: have a return type
 - Return a value of a specific data type using the **return** statement.
- Void functions: do not have a return type
 - Do not use a return statement to return a value

هذا النوع لا يرجع بقيمة ولا يستخدم . **return**

Value-Returning Functions

- Because the value returned by a value-returning function is unique, we must:
 - Save the value for further calculation
 - Use the value in some calculation
 - Print the value

القيمة اللي برجعها ممكن نخزنها في variable أو نستخدمها في عملية حسابية أو نطبعها مثلا.


```

int abs(int number)
int abs(int number)
{
    if (number < 0)
        number = -number;

    return number;
}

```

المثال يبين value-returning function يأخذ رقم ويرجع قيمته المطلقة.

- Heading: first four properties above

– Example: int abs(int number)

data type القيمة التي يرجعها + اسم ال function + قائمة ال parameters.

- Formal Parameter: variable declared in the heading

– Example: number

ال variables التي يخزن فيها القيم التي نرسلها لل function المعرفة في ال . parameter list

- Actual Parameter: variable or expression listed in a call to a function

– Example: x = pow(u, v)

استدعاء ال function في أي function غيره.

Syntax: Value-Returning Function

```

functionType functionName(formal parameter list)
{
    statements
}

```

- `functionType` is also called the data type or return type.

Syntax: Formal Parameter List

```
dataType identifier, dataType identifier, ...
```

example: (`int` number , `int` power)

Function Call

```
functionName(actual parameter list)
```

لما بدنا نستدعي ال `function` في أي `function` ثاني : اسم ال `function` (القيم الي بدنا نرسلها ممكن تكون `expression` و ممكن تكون `variable`).

`abs (2)` : `expression`

`abs(x)` : `variable`

- Formal parameter list can be empty:

```
functionType functionName()
```

ممكن تكون ال `parameter list` فاضية يعني هذا ال `function` ما بحتاج قيم نرسلها اله، كالاتي بنستدعيه:

```
functionName()
```

return Statement

- Once a value-returning function computes the value, the function returns this value via the **return** statement.

فور حساب القيمة الي بدو يرجعها ال function يرجع بالقيمة من خلال . **return** statement

Syntax: **return** Statement

```
return expr;
```

- When a return statement executes:
 - Function immediately terminates
 - Control goes back to the caller

عندما تنفذ **return** statement ينتهي التنفيذ في ال function ويعود الى موقع ال function call.

- When a **return** statement executes in the function main, the program terminates.

عندما تنفذ جملة **return** في ال main function يتوقف البرنامج كله عن العمل، لذلك كما ذكرنا سابقا يجب عند الانتهاء من كتابة البرنامج استخدام **return 0**

```
double larger(double x, double y)
{
    double max;

    if (x >= y)
        max = x;
    else
        max = y;

    return max;
}
```

You can also write this function as follows:

```
double larger(double x, double y)
{
    if (x >= y)
        return x;
    else
        return y;
}
```

```
double larger(double x, double y)
{
    if (x >= y)
        return x;

    return y;
}
```

جملة **return** ممكن أن تستخدم أي مكان داخل الfunction .

Function Prototype

- Function prototype: function heading without the body of the function:

```
functionType functionName(parameter list);
```

- It is not necessary to specify the variable name in the parameter list
- The data type of each parameter must be specified

ليس من الضرورة كتابة اسماء ال variables لكن من الضروري كتابة ال data type لكل parameter.

example: **int** abs (**int** number)

example: **int** max (**int** , **int**)

بايش ممكن يفيدنا ال prototype ؟ مثلا كتبنا function تحت ال main function في هذه الحالة ما بقدر استدعيه في ال main لأنه كما ذكرنا سابقا انه التنفيذ يتم line by line ، الحل ان نضع prototype لهذا ال function t فوق ال main عشان الكومبايلر يعرف انه في تحت function ولما نستدعيه يروح عليه.

الأثر الجميل يعمل بصمت

```

//Program: Largest of three numbers

#include <iostream>

using namespace std;

double larger(double x, double y);
double compareThree(double x, double y, double z);

int main()
{
    double one, two; //Line 1

    cout << "Line 2: The larger of 5 and 10 is "
          << larger(5, 10) << endl; //Line 2

    cout << "Line 3: Enter two numbers: "; //Line 3
    cin >> one >> two; //Line 4
    cout << endl; //Line 5

    cout << "Line 6: The larger of " << one
          << " and " << two << " is "
          << larger(one, two) << endl; //Line 6

    cout << "Line 7: The largest of 23, 34, and "
          << "12 is " << compareThree(23, 34, 12)
          << endl; //Line 7

    return 0;
}

double larger(double x, double y)
{
    if (x >= y)
        return x;
    else
        return y;
}

double compareThree (double x, double y, double z)
{
    return larger(x, larger(y, z));
}

```

Sample Run: In this sample run, the user input is shaded.

Line 2: The larger of 5 and 10 is 10

Line 3: Enter two numbers: 25 73

Line 6: The larger of 25 and 73 is 73

Line 7: The largest of 23, 34, and 12 is 34

لاحظوا انه ال function اللي اسمه larger تعريفه موجود بعد ال main فكتبنا ال prototype الخاص فيه قبل ال main ، نفس الاشئ ل function compareThree .

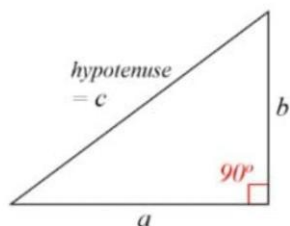
Flow of Execution

- Execution always begins at the first statement in the function main
- Other functions are executed only when they are called
- Function prototypes appear before any function definition
 - The compiler translates these first
- The compiler can then correctly translate a function call

التنفيذ دائما يبدأ من ال main function ، باقي ال functions يتم تنفيذهم فقط اذا استدعيناها، ال prototypes دائما يظهرها قبل التعريف لل functions ليستطيع الكومبايلر أن ينفذ ال function call بشكل صحيح.

Problem-Solving Part

1. Write a C++ program that inputs the a and b sides of the triangle and define a function to calculate the side c of the triangle (the hypotenuse).



$$c^2 = a^2 + b^2$$

2. Write a C++ program that inputs a number and define a function that calculates the factorial of the number.

3. Write a C++ program that inputs the radius of a circle and the width and the length of a rectangle and define two functions to calculate the circle area and the area of the rectangle.

1.

```
#include <iostream>
#include <cmath>
using namespace std;

double hypo(double x, double y)
{
    double z = sqrt(pow(x,2) + pow(y,2));
    return z;
}
int main()
{
    double a,b,c;
    cin>>a>>b;
    c=hypo (a,b);
    cout<<"The hypotenuse is: "<<c;
    return 0;
}
```

2. الأثر الجميل يعمل بصحة

```

#include <iostream>
using namespace std;

int fact (int num)
{
    int i = 1;
    for(int j=num; j>=1; j--)
        i*=j;
    return i;
}
int main()
{
    int number, factorial;
    cout<<"Enter an integer";
    cin>>number;
    factorial = fact(number);
    cout<<"The factorial of the integer you have entered is: "<<factorial;

    return 0;
}

```

3.

```

#include <iostream>
using namespace std;

double circle_area(int r)
{
    double const pi =3.14;
    double area = pi *r*r;
    return area;
}
int rectangle_area(int x, int y)
{
    return x*y;
}

int main()
{
    int radius,width,length,a2;
    double a1;
    cout<<"Enter the radius of the circle"<<endl;
    cin>>radius;
    a1 = circle_area (radius);
    cout<<"The area of the circle is: "<<a1<<endl;
    cout<<"Enter the width and length of the rectangle"<<endl;
    cin>>width>>length;
    a2= rectangle_area (width,length);
    cout<<"The area of the rectangle is: "<<a2;

    return 0;
}

```

مبارره



!حلال



الأثر الجميل يعمل بصحة